

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CARRERA DE Departamento Académico de Ingeniería



**Optimización del tráfico urbano en Lima con
semáforos inteligentes mediante Aprendizaje por
Refuerzo (RL) y simulación en SUMO**

TRABAJO DE INVESTIGACIÓN

Para optar el grado de Bachiller en Ingeniería en Ciencia de la
Computación

AUTOR(ES)

Fabian Martín Alvarado Vargas 

Neftalí Calixto Rojas 

ASESOR(ES)

José Antonio Fiestas Iquiria 

Lima - Perú

2025

Dedicatoria:

[illegible]

Agradecimientos:

[illegible]

TABLA DE CONTENIDO

	Pág.
RESUMEN	1
ABSTRACT	2
INTRODUCCIÓN	3
Presentación del tema de investigación	3
Descripción de la situación problemática	3
Formulación del problema	3
Objetivos de investigación	4
Justificación	5
Alcance y limitaciones / restricciones	5
Áreas de investigación y herramientas	6
Métricas de evaluación	6
Arquitectura modular propuesta (visión general)	7
Plan de trabajo (breve)	7
 CAPÍTULO I ESTADO DEL ARTE Y REVISIÓN CRÍTICA DE LA LITERATURA	 8
1.1 Panorama general	8
1.2 Estado del arte: trabajos estrechamente relacionados	8
1.3 Revisión crítica y vacíos identificados	10
1.4 Síntesis comparativa	12
1.5 Implicancias para el diseño de esta tesis	12

CAPÍTULO II MARCO TEÓRICO	14
2.1 Arquitectura teórica del sistema (visión por módulos)	15
2.2 Definiciones y conceptos fundamentales	17
2.3 Relación con objetivos y resultados esperados	19
CAPÍTULO III Marco metodológico	20
3.1 Alcance y unidades	20
U1: Preprocesamiento	20
U2: Entrenamiento	20
3.2 Requisitos de hardware	21
3.3 Instalación y configuración de SUMO	21
3.4 Limpieza y configuración del escenario OSM $\rightarrow$ SUMO	22
3.5 Importación de un cruce desde OSM	27
3.5.1 Asistente OSM (<i>osmWebWizard</i>)	27
3.6 Generación y ejecución del escenario	30
3.6.1 Generar rutas de vehículos	30
3.6.2 Crear el archivo de configuración	30
3.6.3 Ejecución de SUMO	31
3.7 Diagrama de flujo del módulo U1	31
3.8 Resumen de las unidades	33
3.9 Desarrollo del módulo U2: Aprendizaje por Refuerzo para el control de semáforos	33
CONCLUSIONES	41
RECOMENDACIONES	42
ANEXOS	46

ÍNDICE DE TABLAS

Tabla 1.1	Comparación de trabajos relacionados con control/optimización semafórica.	12
Tabla 3.1	Recursos recomendados para construir y ejecutar el escenario. .	21

ÍNDICE DE FIGURAS

Figura 2.1	Diagrama de diseño del proyecto: U1 (preprocesamiento), U2 (RL), U3 (simulación/medición en SUMO) y U4 (coordinación distribuida).	15
Figura 3.1	Vista inicial en <code>netedit</code> con la intersección importada; hay calles secundarias y direcciones de carril incorrectas.	24
Figura 3.2	Misma intersección tras la limpieza: calles irrelevantes eliminadas y sentidos de los carriles corregidos.	24
Figura 3.3	Antes y después de la limpieza de la red en <code>netedit</code> .	24
Figura 3.4	Selección del <i>bounding box</i> en OSM mediante el asistente <i>osm-Web Wizard</i> .	29
Figura 3.5	Red del cruce generada automáticamente en <i>sumo-gui</i> .	29
Figura 3.6	Importación de un cruce desde OSM a SUMO.	29
Figura 3.7	Diagrama de flujo del módulo U1: preprocesamiento y generación del escenario SUMO.	32
Figura 3.8	Diagrama de flujo del módulo U2: entrenamiento del agente de Aprendizaje por Refuerzo para el control semafórico.	40

RESUMEN

En Lima Metropolitana existen puntos críticos de congestión vehicular, especialmente en intersecciones y corredores urbanos de alto tráfico. Esta investigación propone **reducir el retraso promedio por vehículo** mediante **semaforización adaptativa** con **Aprendizaje por Refuerzo (RL)** y validación en **SUMO**.

La solución consiste en un *pipeline* reproducible con cuatro unidades: (U1) datos y preprocesamiento; (U2) agentes de **RL** (un agente por intersección); (U3) simulación y medición en SUMO; y (U4) **coordinación distribuida entre semáforos**. En U4, cada agente comparte mensajes ligeros con sus *vecinos* (fase actual, colas previstas, salida esperada) para **coordinar ondas verdes** y **evitar spill-back**. Se implementarán y compararán **Q-learning, SARSA, DQN y PPO**; y, para coordinación, enfoques multiagente (IPPO/MAPPO) con **estado extendido** y **recompensa con término de coordinación**.

Las métricas de salida incluyen: *delay* y tiempo de viaje, colas (promedio/máxima), *throughput* (veh/h), paradas/vehículo, emisiones (CO₂, NO_x) y recompensa acumulada. Como metas indicativas en simulación: **25–40 % de reducción del retraso** frente a control fijo y **≥ 20 % de reducción de colas**, con mejoras adicionales al activar U4.

Palabras clave:

semáforos inteligentes; aprendizaje por refuerzo; coordinación distribuida; SUMO; Lima Metropolitana; control de tráfico

ABSTRACT

Optimización del tráfico urbano en Lima mediante semáforos inteligentes con Aprendizaje por Refuerzo (RL) y simulación en SUMO

Lima Metropolitana presenta congestión severa en intersecciones y corredores de alto flujo, lo que incrementa los tiempos de viaje, las emisiones y reduce la movilidad. Esta tesis busca **reducir el retraso promedio por vehículo** mediante **control semafórico adaptativo** basado en **Aprendizaje por Refuerzo (RL)**, validado en el simulador **SUMO**.

Se propone un *pipeline* reproducible con cuatro unidades: (U1) preprocesamiento de datos y generación de red/rutas en SUMO; (U2) **RL** (entorno Gym+TraCI); (U3) simulación y medición en SUMO; y (U4) **coordinación distribuida entre semáforos**, donde cada intersección es un agente que *se comunica* con sus vecinas para crear *ondas verdes* y evitar *spillback*. En U2 se implementarán y compararán **Q-learning, SARSA, DQN y PPO**; en U4 se evaluarán variantes multiagente (IPPO/MAPPO) con **estado extendido** (información de vecinos) y **recompensa con penalización por spillback**.

Las métricas incluyen: retraso y tiempo de viaje, longitud de colas, *throughput* (veh/h), número de paradas, emisiones estimadas (CO₂, NO_x) y recompensa acumulada. Metas indicativas en simulación: **25–40 % menos retraso** y **≥20 % menos colas** frente a control fijo, y **mejoras adicionales** cuando se activa U4.

Palabras clave:

semáforos inteligentes; aprendizaje por refuerzo; control multiagente; coordinación distribuida; SUMO; movilidad urbana

INTRODUCCIÓN

Presentación del tema de investigación

La congestión vehicular en Lima Metropolitana afecta de manera crítica la movilidad urbana, el tiempo de viaje y la calidad del aire. Este trabajo propone **optimizar el control semafórico** de una intersección (o pequeño corredor) de alto tráfico en Lima mediante **Aprendizaje por Refuerzo (RL)** y validación en el simulador **SUMO**. Se adopta un enfoque *multiagente* (un agente por intersección) y una evaluación rigurosa frente a esquemas tradicionales (fijo y actuado). El estudio se centra en el desarrollo de un *pipeline* reproducible de datos $\rightarrow$ preprocesamiento $\rightarrow$ SUMO $\rightarrow$ RL $\rightarrow$ evaluación.

Descripción de la situación problemática

En intersecciones y corredores urbanos de alto flujo se originan colas extensas, tiempos de espera elevados y variabilidad temporal significativa (picos laborales/escolares, eventos, incidentes). Los controladores semafóricos de *tiempo fijo* y varios esquemas *actuados* no capturan adecuadamente esta dinámica, lo que limita su efectividad. La consecuencia es un aumento del *delay* por vehículo, mayor número de paradas, menor *throughput* y un incremento de emisiones contaminantes locales.

Formulación del problema

Problema central: Crear un modelo que reduzca el **retraso promedio por vehículo** en un punto de alto tráfico en Lima Metropolitana mediante semaforización adaptativa basada en **RL**, y validarlo en **SUMO**.

Metas cuantificables:

- Reducción del retraso promedio por vehículo en **25–40 %** respecto a control fijo.
- Disminución de la longitud promedio de colas en **20 %**.
- Incremento del *throughput* en **10 %**.
- Reducción de emisiones (CO₂) en **5 %** (estimación SUMO).

Objetivos de investigación

Objetivo general: Diseñar y evaluar un **modelo de control de semáforos** basado en **RL**, con **coordinación distribuida** entre intersecciones, validado en **SUMO** para reducir tiempos de espera y colas en Lima Metropolitana.

Objetivos específicos:

- (U1) Recopilar y preprocesar datos de tráfico y convertirlos a formato SUMO (`.net.xml`, `.rou.xml`).
- (U2) Diseñar estados, acciones y recompensas para agentes de RL que controlen fases semaforicas; implementar y comparar **Q-learning**, **SARSA**, **DQN** y **PPO**.
- (U3) Validar en SUMO con métricas: *delay*, colas, *throughput*, paradas y emisiones.

- (U4) Implementar *coordinación distribuida* entre semáforos: añadir una **capa de mensajería** entre agentes vecinos (fase actual, colas/salidas previstas), un **estado extendido** que incorpore información de vecinos y una **recompensa con penalización por *spillback***. Evaluar variantes multiagente (IPPO/MAPPO) y su impacto a nivel de *corredor*.
- Proponer buenas prácticas para reproducibilidad (scripts, semillas, análisis estadístico con IC95 %).

Justificación

Social y económica: reducción de tiempos de viaje y variabilidad, mayor productividad y menor gasto de combustible. **Ambiental:** menor número de paradas y de tiempos de ralentí, con potencial disminución de CO₂ y NOx. **Académica y tecnológica:** aplicación de técnicas de **RL multiagente** sobre un caso realista de Lima, con comparación sistemática frente a semaforización fija/actuada y un *pipeline* abierto y reproducible.

Alcance y limitaciones / restricciones

Alcance:

- Modelado de un tramo representativo de intersecciones en Lima Metropolitana.
- Entrenamiento de agentes RL (multiagente, un agente por intersección) en SUMO con escenarios de hora punta y valle.
- Comparación con semaforización fija y actuada.

No incluido en esta fase:

- Implementación en hardware real de semáforos.
- Cobertura completa de toda la red vial de Lima.

Áreas de investigación y herramientas

- **Ingeniería de transporte:** SUMO, OpenStreetMap, métricas de tráfico.
- **Inteligencia Artificial:** Python, `stable-baselines3` (PPO, DQN), PyTorch.
- **Procesamiento de datos:** Python (`pandas`, `numpy`), SUMO tools (`netconvert`, `duarouter`).
- **Modelado y simulación:** SUMO + TraCI; Docker (opcional).
- **Estadística y visualización:** `scipy`, `matplotlib`; pruebas de comparación.

Métricas de evaluación

- Tiempo promedio de viaje y de espera por vehículo.
- Longitud de colas (promedio y máxima) por intersección.
- *Throughput* (vehículos/hora).
- Número de paradas por vehículo.
- Emisiones estimadas (CO₂, NO_x, consumo de combustible) según SUMO.
- Recompensa acumulada y curvas de convergencia de los algoritmos RL.

Arquitectura modular propuesta (visión general)

Se adopta una arquitectura con cuatro unidades: **U1** Preprocesamiento (datos $\rightarrow$ SUMO), **U2** RL (entorno Gym + agentes y entrenamiento), **U3** SUMO (simulación y medición) y **U4 Coordinación distribuida**.

Flujo: U1 alimenta a U3 (red y rutas); U3 expone estados y métricas vía TraCI; U2 decide acciones (fases) que se aplican en U3. En U4, cada agente intercambia *mensajes* con sus intersecciones vecinas (p.ej., *fase actual*, *tiempo restante*, *outflow estimado hacia el vecino*, *colas en salidas*) y **amplía su estado** con un *resumen de vecinos* para coordinar *ondas verdes* y evitar *spillback*.

Plan de trabajo (breve)

1. Recolección y preprocesamiento de datos (2–4 semanas).
2. Modelado en SUMO y escenario base (2–3 semanas).
3. Implementación del entorno Gym + TraCI y agentes RL (3–6 semanas).
4. Entrenamiento y experimentos comparativos (4–8 semanas).
5. Análisis estadístico, redacción de resultados y recomendaciones (3–4 semanas).

CAPÍTULO I

ESTADO DEL ARTE Y REVISIÓN CRÍTICA DE LA LITERATURA

1.1 Panorama general

El control adaptativo de semáforos con Aprendizaje por Refuerzo (RL) se ha consolidado como una alternativa a la semaforización fija y actuada, al permitir políticas que se adaptan en tiempo real a la variación de flujos. La literatura reciente estudia tanto agentes únicos por intersección (single-agent) como enfoques multiagente (un agente por intersección) y evalúa su impacto en métricas como retraso promedio, colas, throughput y emisiones. En esta sección se sintetizan tres trabajos directamente relevantes a nuestro proyecto (Uniandes 2023, Trinity 2022, UAQ 2024) y se discuten sus aportes, limitaciones y la pertinencia para Lima Metropolitana.

1.2 Estado del arte: trabajos estrechamente relacionados

Universidad de los Andes (2023): *Semáforos inteligentes por medio de aprendizaje por refuerzo* [1]

Objetivo y configuración. Estudiar el desempeño de algoritmos RL (Q-learning, SARSA, DQN y PPO) para reducir tiempos de espera frente a un control de tiempos fijos en un escenario simulado.

Métodos. Implementación de algoritmos tabulares (Q-learning, SARSA) y de Deep

RL (DQN, PPO) con política ϵ -greedy en tabulares y objetivos recortados en PPO; comparación contra baseline de tiempos fijos.

Métricas. Eficiencia relativa frente a tiempos fijos (reducción del tiempo de espera), número de vehículos en sistema, estabilidad de la política.

Resultados. Reporta eficiencias de **35.79 % (Q-learning)**, **51.63 % (SARSA)**, **63.40 % (PPO)** y **63.49 % (DQN)** frente a control fijo. DQN y PPO muestran mejor adaptación y menor tiempo de espera, con DQN muy ligeramente superior en el caso analizado.

Limitaciones. Demanda y geometría sintetizadas; no integra datos reales de conteo/OD ni perturbaciones estocásticas complejas; sin análisis estadístico robusto (p. ej., ICs o pruebas de significancia).

Lecciones para esta tesis. (i) Incluir *deep baselines* (DQN/PPO) junto a tabulares; (ii) reportar mejoras con métricas comparables; (iii) añadir pruebas de significancia y sensibilidad.

Trinity College Dublin (2022): *Reinforcement Learning for Traffic Light Optimization* [2]

Objetivo y configuración. Comparar agentes fijos/aleatorios con **DQN**, **A2C** y **PPO** en casos single-agent, multi-agent y un caso más realista.

Métodos. SUMO+TraCI, definición de estados (colas/velocidades/ocupación), espacios de acción (fases/tiempos) y recompensas (tiempos de espera acumulados/negativos).

Métricas. **Tiempo de espera acumulado** por intersección vs. pasos de simulación; comparación entre agentes y escenarios.

Resultados. **PPO** es consistentemente el mejor, seguido de **A2C**, agentes fijos/aleatorios; **DQN** queda por detrás en sus pruebas.

Limitaciones. Resultados centrados en comparativa algorítmica; no se usan datos

reales de OD, ni se reportan pruebas de significancia.

Lecciones para esta tesis. (i) PPO y A2C son fuertes candidatos para entornos urbanos; (ii) conviene incorporar escenarios multiagente; (iii) sumar análisis estadístico y casos con datos reales para mayor validez externa.

UAQ (2024): *Predicción de tráfico vehicular aplicada a la optimización semafórica en un microsimulador* [3]

Objetivo y configuración. Integrar **predicción de demanda** (AR, Filtros de Kalman, RNN) con **microsimulación** para **optimizar ciclos semafóricos** en un corredor real.

Datos. Históricos de **7 intersecciones** (ago–sep 2022), con escenarios de 1 hora, 1 día, 1 semana y 1 mes.

Métricas. Errores de predicción (RMSE, MAE, MAPE, R^2) y, tras aplicar los ciclos en el simulador, tiempo de recorrido y colas.

Resultados. **RNN y Filtros de Kalman** alcanzan $R^2 \approx 0.95$ en horizontes corto/medio; al aplicar los ciclos óptimos, se obtiene **hasta 36.34 % de reducción del tiempo de viaje**.

Limitaciones. Optimización semafórica basada en pronóstico (no en RL); generalización a otras franjas/variabilidad no evaluada; requiere calidad de datos.

Lecciones para esta tesis. (i) Incorporar **datos reales (OD, aforos)** y **predicción de demanda** como *módulo upstream* que alimente al agente RL; (ii) evaluar impacto sobre **tiempos de viaje** además del retraso.

1.3 Revisión crítica y vacíos identificados

Modelado de la realidad. Uniandes y Trinity validan con escenarios simulados; UAQ sí usa datos reales, pero sin RL. Nuestra tesis puede *combinar* ambas

fortalezas: **RL + datos reales (OD/aforos) + SUMO**.

Comparación algorítmica. Trinity muestra ventajas de PPO/A2C; Uniandes añade DQN con buen rendimiento. Probaremos **PPO, A2C y DQN** (y tabuladores como línea base) en *escenario limeño*.

Predicción de demanda. UAQ evidencia el valor de pronósticos robustos (RN-N/Kalman). Propondremos un **módulo opcional de demanda** para alimentar la simulación y evaluar *RL con y sin pronóstico*.

Rigor experimental. Faltan análisis de significancia y sensibilidad. Nuestra evaluación incluirá **intervalos de confianza**, pruebas (p. ej., Mann–Whitney) y **ablation studies** (estado/acción/recompensa).

Transferencia a Lima. Escogeremos una intersección/corredor representativo de Lima, con escenarios *punta/valle*, e integraremos **multiagente** (un agente por intersección) y **restricciones operativas** (tiempos mínimos de verde/ámbar/rojo, coordinación en arterias).

1.4 Síntesis comparativa

Trabajo	Datos	Simulador	Algoritmos	Resultados clave
Uniandes (2023) [1]	Sintéticos	SUMO	Q-learning, SARSA, DQN, PPO	Eficiencias vs. fijo: 35.79 %, 51.63 %, 63.40 %, 63.49 %
Trinity (2022) [2]	Sintéticos	SUMO	DQN, A2C, PPO	PPO > A2C > (fijo/aleatorio) > DQN en tiempo de espera
UAQ (2024) [3]	Reales (7 inters.)	SUMO	AR, Kalman, RNN + optimización de ciclos	$R^2 \approx 0.95$ (corto/-medio); -36.34 % en tiempo de viaje

TABLA 1.1: Comparación de trabajos relacionados con control/optimización semafórica.

1.5 Implicancias para el diseño de esta tesis

A partir del estado del arte, nuestra tesis implementará: (i) un entorno **SUMO+TraCI** con **agentes multiagente** (un TSL por intersección), (ii) **PPO, A2C y DQN** como algoritmos principales (más Q-learning/SARSA como líneas base), y (iii) evaluación rigurosa (delay, colas, *throughput*, emisiones) con pruebas estadísticas y análisis de sensibilidad.

Aporte específico (U4: Coordinación distribuida). Se incorpora un módulo de **coordinación distribuida entre semáforos** para pasar de optimizaciones locales a **optimización de corredor**:

- *Estado extendido*: cada agente observa su entorno local (colas por carril, fase, tiempo en fase) y un **resumen de vecinos** (fase vecina, tiempo restante, estimación de *outflow* hacia su enlace compartido).
- *Mensajería ligera*: canal de comunicación simulado (latencia/pérdida configurables) para intercambiar mensajes agent-to-agent (*fase, outflow esperado, colas en salidas*).
- *Recompensa con coordinación*: término explícito de **spillback** para evitar que una cola bloquee a la intersección vecina; opcionalmente mezcla con una señal global (R_{global}).
- *Algoritmos*: **IPPO** (PPO independiente con pesos compartidos) como base; extensión a **MAPPO** (crítico centralizado) si el tiempo lo permite.
- *Evaluación a nivel corredor*: tiempo de viaje extremo a extremo, *throughput* total, longitud de cola máxima y tasa de *spillback*, además de métricas locales.

Este U4 añade el tópico de **Sistemas Distribuidos** (protocolos de intercambio, latencia, tolerancia a fallos) al marco de RL, incrementa el esfuerzo computacional (entrenamiento multiagente con estado ampliado) y permite demostrar beneficios de **coordinación** frente a agentes aislados, alineado con la literatura reciente en control multiagente.

CAPÍTULO II

MARCO TEÓRICO

El presente capítulo desarrolla los conceptos fundamentales que sustentan la arquitectura del proyecto y explican por qué cada módulo propuesto aporta a la reducción del retraso promedio por vehículo en intersecciones urbanas. A diferencia del diagrama metodológico del capítulo siguiente (centrado en el flujo operativo), aquí el foco es *arquitectural*: qué módulos lo componen, qué principios teóricos utiliza cada uno y qué resultados espera producir.

En particular, se definen los elementos teóricos que habilitan: (i) la construcción del entorno de simulación en SUMO [4, 5], (ii) el control semafórico adaptativo mediante Aprendizaje por Refuerzo (RL) con algoritmos tabulares y de *Deep RL* [6–8], (iii) la instrumentación de métricas de movilidad urbana [9–11] y (iv) la coordinación distribuida multiagente entre semáforos para optimizar a nivel de corredor [12].

2.1 Arquitectura teórica del sistema (visión por módulos)

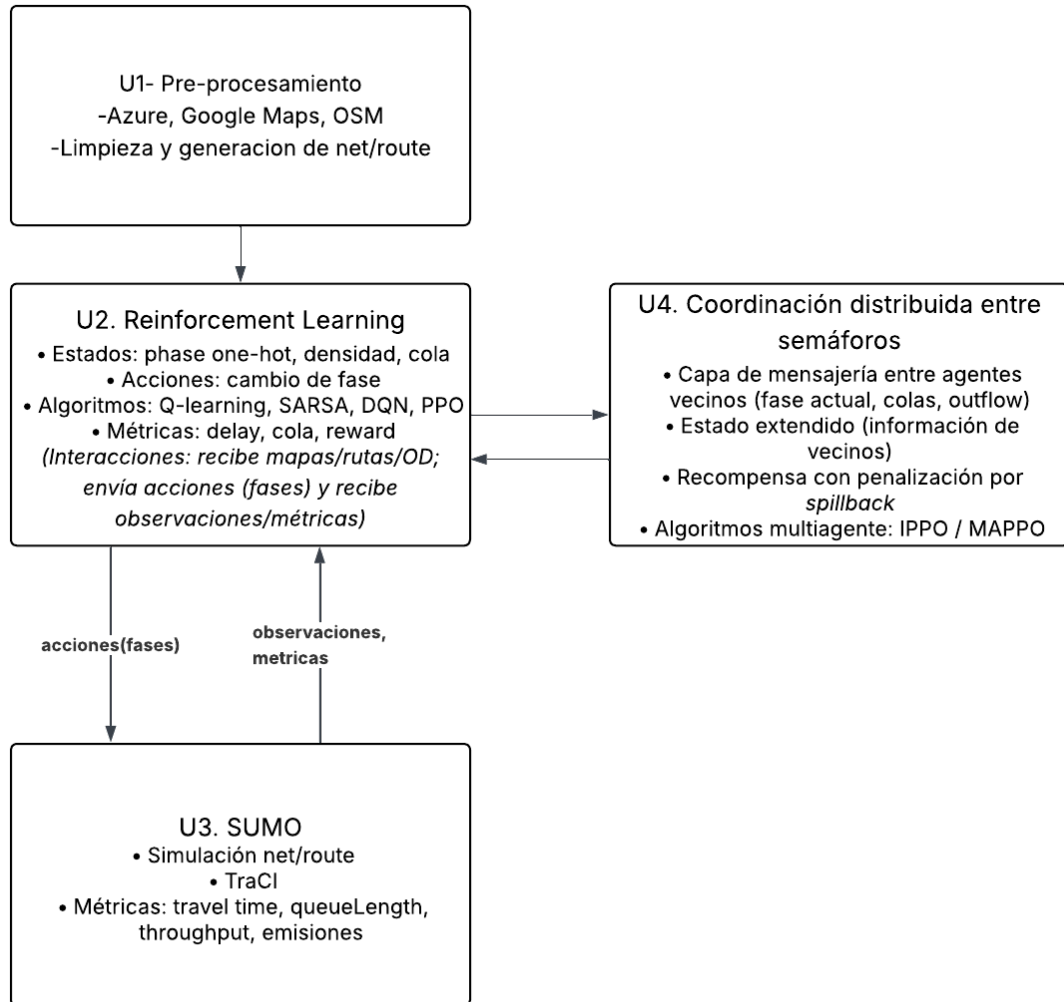


FIGURA 2.1: Diagrama de diseño del proyecto: U1 (preprocesamiento), U2 (RL), U3 (simulación/medición en SUMO) y U4 (coordinación distribuida).

U1. Preprocesamiento y generación del entorno SUMO. Se encarga de transformar una intersección o pequeño corredor urbano en una red SUMO utilizable: conversión de cartografía (OSM $\rightarrow$ `.net.xml`), definición de flujos y rutas (`.rou.xml`), y configuración base de semáforos (`.add.xml`). Teóricamente, se apoya

en la *microsimulación* vehicular y en la representación de grafos viales donde nodos son intersecciones y aristas son tramos con uno o más carriles y restricciones de giro [4, 5].

U2. Control semafórico con Aprendizaje por Refuerzo (RL). Modela cada semáforo como un agente que observa el estado del entorno (colas, fase actual, tiempos de espera), selecciona una acción (mantener, conmutar, extender fase) y recibe una recompensa que guía el aprendizaje. Se consideran algoritmos tabulares (Q-learning, SARSA) y de *Deep RL* (DQN, PPO). Los elementos teóricos clave son: espacio de estados, espacio de acciones, función de recompensa, política, episodio e interacción agente-entorno [6–8].

U3. Simulación y medición en SUMO (TraCI). SUMO provee la dinámica del tráfico a nivel de vehículo; mediante TraCI se extraen variables relevantes (velocidades, tiempos de espera, colas) y se aplican las acciones decididas por los agentes. Teóricamente, este módulo sostiene la *retroalimentación* RL: estados y recompensas se computan a partir de la simulación paso a paso; las métricas de evaluación (delay, longitud de colas, *throughput*, paradas, emisiones) se definen aquí con base en la literatura de ingeniería de transporte [9–11, 13–15].

U4. Coordinación distribuida entre semáforos (multiagente). Extiende el RL local hacia la cooperación entre intersecciones vecinas mediante una capa ligera de mensajería (fase actual, *outflow* esperado, colas en salidas), un *estado extendido* (información propia + resumen de vecinos) y una recompensa con término de penalización por *spillback*. En términos teóricos, se apoya en *RL multiagente* y *sistemas distribuidos* (comunicación asíncrona, latencias, tolerancia a fallos), con variantes como IPPO/MAPPO para el aprendizaje coordinado [12].

2.2 Definiciones y conceptos fundamentales

SUMO (Simulation of Urban MObility)

Microsimulador de código abierto que modela el movimiento de vehículos a nivel individual sobre una red vial dirigida. Permite definir redes, rutas, tipos de vehículo y control semafórico, y calcular métricas de desempeño (tiempo de viaje, colas, *throughput*, emisiones) [14, 15].

TraCI (Traffic Control Interface)

Interfaz cliente-servidor que conecta SUMO con programas externos (p. ej., Python), habilitando control en tiempo de simulación: lectura de observaciones (colas por carril, fase activa, tiempos de espera) y aplicación de acciones (cambios de fase, duraciones) [13, 14].

Aprendizaje por Refuerzo (RL)

Paradigma de aprendizaje donde un agente interactúa con un entorno para maximizar recompensa acumulada. Elementos: *estado* (s), *acción* (a), *recompensa* (r), *política* (π) y *episodio*. En control semafórico, el objetivo es minimizar retrasos y colas penalizando paradas y *spillback* [6].

Q-learning y SARSA (RL tabular)

Métodos que aproximan el valor de acción $Q(s, a)$ en tablas discretas. Q-learning es *off-policy*; SARSA es *on-policy*. Se recomiendan como líneas base por su sencillez e interpretabilidad en espacios de estado discretizados [6].

DQN y PPO (Deep RL)

DQN aproxima $Q(s, a)$ con redes neuronales profundas (*value-based*); PPO optimiza directamente la política con restricciones de probabilidad (*policy-based*). Son adecuados para espacios de estado continuos o de mayor dimensionalidad [7, 8, 16].

RL multiagente: IPPO / MAPPO

En IPPO cada agente entrena su política de forma independiente (posibles pesos compartidos); MAPPO introduce un crítico centralizado para estabilidad en entornos cooperativos. Aplicable a corredores donde varias intersecciones deben coordinar [12].

Métricas de movilidad

Delay/tiempo de espera: diferencia entre tiempo real y tiempo libre de congestión. **Longitud de cola:** número de vehículos detenidos/ralentizados por carril. **Throughput:** vehículos/hora que atraviesan una sección. **Paradas/vehículo:** número de detenciones. **Emisiones:** estimadas por modelo de SUMO a partir de aceleraciones/velocidades [9–11, 15].

Spillback y ondas verdes

Spillback: cola que rebasa la intersección y bloquea enlaces adyacentes; debe penalizarse en la recompensa para evitar colapsos. *Ondas verdes:* alineación temporal de fases para favorecer flujos dominantes a lo largo de un corredor [9, 11].

Reproducibilidad y validez

Semillas aleatorias, scripts de ejecución y reporte de IC95 % en métricas clave refuerzan la robustez de resultados y permiten comparación entre configuraciones (con/sin coordinación, distintos algoritmos). En la práctica, se recomienda documentar configuración y versiones de librerías (p. ej., `stable-baselines3`) [16].

2.3 Relación con objetivos y resultados esperados

La arquitectura teórica se alinea con los objetivos específicos: (U1) producir una red y rutas SUMO reproducibles; (U2) diseñar y entrenar agentes RL (tabulares y *Deep RL*); (U3) medir con rigor las métricas de movilidad; (U4) evaluar coordinación distribuida para beneficios a nivel de corredor (tiempo extremo a extremo, tasa de *spillback*). Los resultados esperados incluyen reducciones de delay y colas frente a control fijo/actuado y mejoras adicionales cuando se activa la coordinación multiagente [9, 11, 12].

CAPÍTULO III

Marco metodológico

Este capítulo presenta de forma concisa los pasos técnicos y los requisitos necesarios para levantar un entorno de simulación de tráfico con SUMO y preparar el escenario que servirá de base para entrenar un agente de aprendizaje por refuerzo. El objetivo es que cualquier lector pueda instalar SUMO, importar una intersección real mediante datos de OpenStreetMap (OSM), generar la red y las rutas vehiculares, y ejecutar la simulación desde la línea de comandos. Se incluye además una descripción breve de las funciones de las unidades U1 (preprocesamiento) y U2 (entrenamiento) y los requisitos de hardware mínimos recomendados.

3.1 Alcance y unidades

U1: Preprocesamiento comprende todas las tareas relacionadas con la construcción del escenario. Esto incluye la instalación y configuración de SUMO, la descarga y conversión de un recorte de OSM en una red SUMO, la generación de rutas básicas de vehículos y la creación de archivos de configuración reproducibles (`.net.xml` para la red, `.rou.xml` para la demanda y `.sumocfg` para el escenario). El resultado de U1 es un escenario listo para simular.

U2: Entrenamiento toma como entrada el escenario de U1 y ejecuta SUMO a través de la interfaz TraCI para que un agente de aprendizaje por refuerzo observe el estado del tráfico y modifique las fases de los semáforos. En U2 no se modifican la red ni las rutas: el enfoque se centra en el algoritmo de control.

3.2 Requisitos de hardware

Para ejecutar U1 y U2 se recomiendan los recursos de la Tabla 3.1. U1 puede ejecutarse en un equipo doméstico modesto, mientras que U2 se beneficia de una GPU si se entrenan modelos complejos.

	Mínimo (U1)	Recomendado (U1+U2)
CPU	4 hilos reales	8–12 hilos
RAM	8 GB	16–32 GB
GPU	no requerida	≥ 6 GB VRAM
Almacenamiento	10 GB SSD	20–50 GB SSD
Sistema operativo	Windows 10/11 o Ubuntu 20.04+	Windows 10/11 o Ubuntu 20.04+

TABLA 3.1: Recursos recomendados para construir y ejecutar el escenario.

3.3 Instalación y configuración de SUMO

1. **Descargar e instalar SUMO.** Obtenga la versión estable de SUMO desde el sitio oficial y siga las instrucciones de instalación para su sistema operativo. SUMO incluye varios ejecutables de consola, como *sumo* (simulación), *sumo-gui* (versión con interfaz gráfica) y *netconvert* (importación de redes). Instale también Python 3, necesario para ejecutar los scripts auxiliares.
2. **Configurar el entorno.** Defina la variable de entorno `SUMO_HOME` apuntando al directorio donde se instaló SUMO y añada la carpeta `bin` al `PATH` del sistema. Esto permite invocar los programas de SUMO desde cualquier terminal.

3. **Verificar la instalación.** Abra una terminal y ejecute los siguientes comandos. Deben devolver la versión instalada de SUMO para confirmar que el entorno está listo:

```
sumo -V  
sumo-gui -V
```

3.4 Limpieza y configuración del escenario OSM $\rightarrow$ SUMO

Tras importar la red desde OSM, es necesario revisar y ajustar el escenario para que represente fielmente la intersección de estudio y resulte adecuado para el entrenamiento del agente de control. Este proceso consiste en eliminar elementos irrelevantes, corregir inconsistencias topológicas y configurar adecuadamente los semáforos y la demanda. A continuación se describe un procedimiento general:

Eliminación y simplificación de la red

1. **Abrir la red en un editor.** Utilice `sumo-gui` o `netedit`, las herramientas de SUMO destinadas a la inspección y edición de redes. Desde la interfaz gráfica se puede seleccionar manualmente cada calle, carril e intersección para examinar su geometría y atributos.
2. **Eliminar calles irrelevantes.** Desactive o borre los segmentos de vía que no formen parte del cruce objeto de estudio. En `netedit` esto puede realizarse en modo “Edge” seleccionando los elementos y pulsando la tecla Suprimir. También se pueden modificar jerarquías de carreteras para mantener únicamente arterias principales y desechar caminos secundarios.

3. **Ajustar sentidos de circulación.** Verifique que las direcciones de cada carril coinciden con la realidad (por ejemplo, si un carril debería ser unidireccional). En **netedit** es posible invertir la dirección de un carril o fusionar dos carriles paralelos empleando la herramienta de edición de lanes.
4. **Simplificar nodos complejos.** Algunas importaciones desde OSM generan nodos excesivamente subdivididos. Estos pueden fusionarse mediante la opción de simplificación de intersecciones en **netconvert** o mediante edición manual. Una red más simple facilita el entrenamiento y reduce tiempos de simulación.

Tras completar estos pasos, la red se limpia y simplifica. La Figura 3.3 muestra un ejemplo antes y después de la limpieza realizada en **netedit** para el cruce estudiado. Se observa cómo se eliminan calles secundarias y se ajusta la orientación de los carriles.

0.48

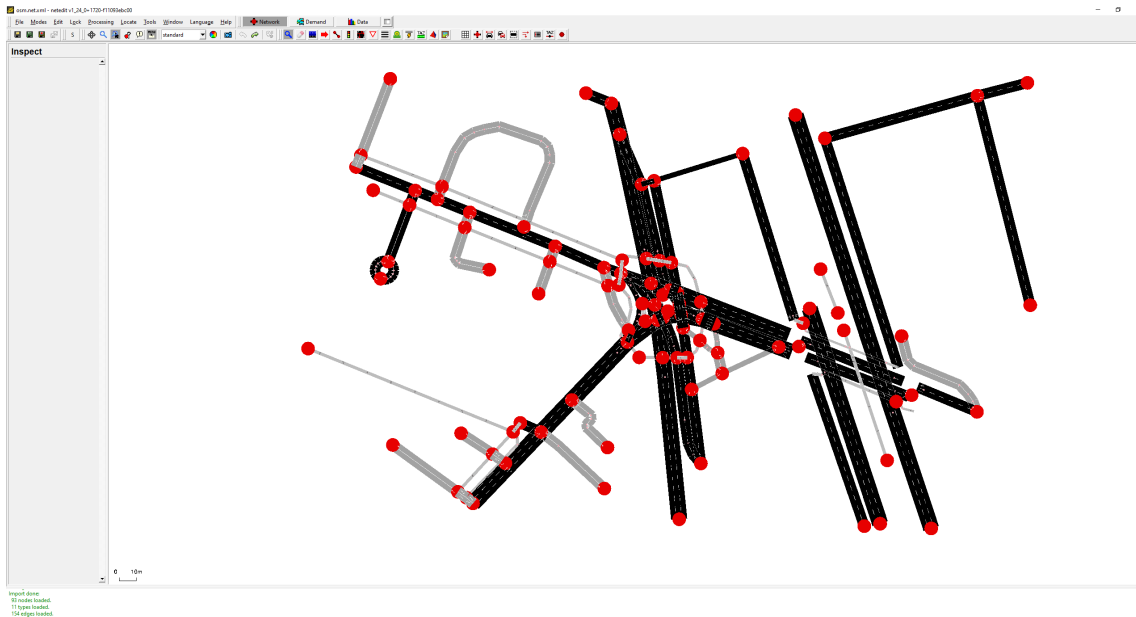


FIGURA 3.1: Vista inicial en **netedit** con la intersección importada; hay calles secundarias y direcciones de carril incorrectas.

0.48

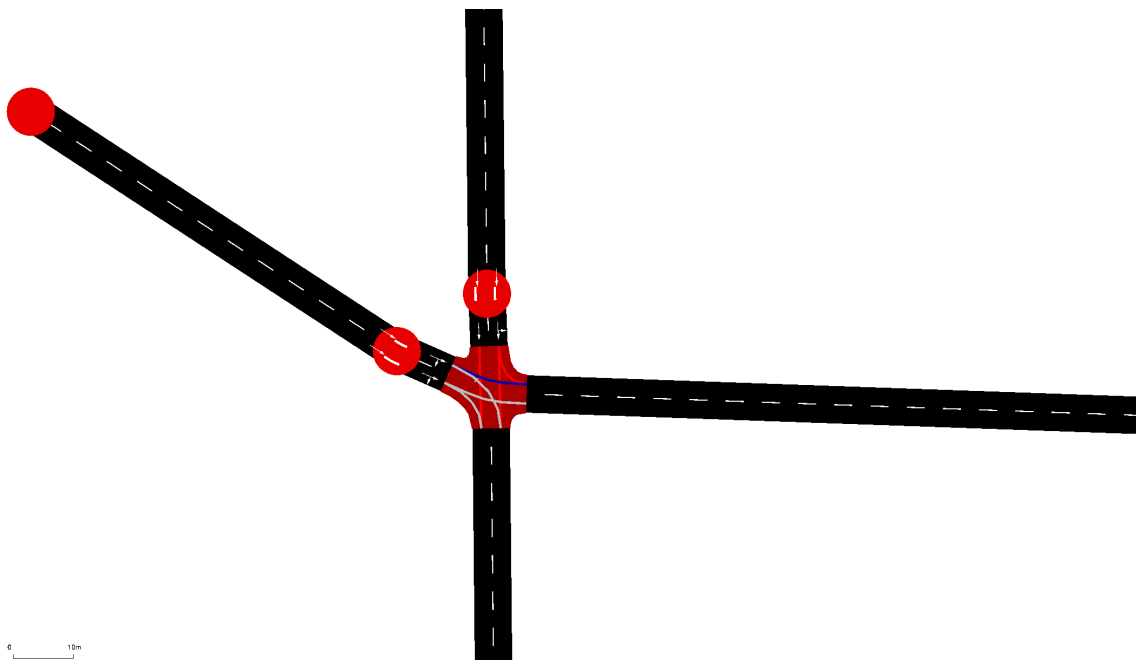


FIGURA 3.2: Misma intersección tras la limpieza: calles irrelevantes eliminadas y sentidos de los carriles corregidos.

FIGURA 3.3: Antes y después de la limpieza de la red en **netedit**.

Configuración de semáforos y demanda

1. **Revisar semáforos importados.** Si al generar el escenario se activó la opción “Traffic lights”, los semáforos aparecerán como nodos `<tlLogic>` en el archivo `.net.xml`. Utilice *sumo-gui* o *netedit* para inspeccionar las fases (colores) y tiempos asignados automáticamente. Ajuste los valores para que respeten las normativas locales: por ejemplo, establezca un mínimo de cuatro segundos de luz ámbar y defina duraciones de luz verde proporcionales al flujo de cada aproximación.
2. **Crear o modificar programas de semáforos.** Para controlar manualmente los semáforos se puede definir un archivo adicional `.add.xml`. Un ejemplo sencillo de un programa estático para una intersección de dos fases es:

```
<additional>
  <tlLogic id="tl_cruce" type="static" programID="base" offset="0">
    <!-- Fase 0: verde en NS, rojo en EW -->
    <phase duration="30" state="GrGr"/>
    <!-- Fase 1: ámbar en todas las aproximaciones -->
    <phase duration="4" state="yryr"/>
    <!-- Fase 2: verde en EW, rojo en NS -->
    <phase duration="30" state="rGrG"/>
    <!-- Fase 3: ámbar en todas las aproximaciones -->
    <phase duration="4" state="ryry"/>
  </tlLogic>
</additional>
```

Este archivo debe referenciarse en el fichero `.sumocfg` dentro del apartado `<additional-files>` para que SUMO lo cargue junto con la red y las rutas.

Si se decide mantener los semáforos generados automáticamente, conviene al menos fijar sus tiempos mínimos y comprobar que las fases son coherentes.

3. **Configurar la demanda.** La generación de rutas con `randomTrips.py` produce una distribución uniforme de viajes sobre la red. Para configurar tasas de llegada realistas conviene ajustar el parámetro `-p` (periodo medio entre vehículos) y usar distintas semillas con `--seed` para reproducibilidad. Si se dispone de datos de tráfico, se pueden definir flujos en un archivo `.rou.xml` indicando el intervalo de tiempo, la tasa de arribo y el tipo de vehículo. Un ejemplo básico de flujo es:

```
<routes>
  <vType id="car" accel="2.6" decel="4.5" length="5" maxSpeed="13.9"/>
  <flow id="f1" begin="0" end="900" vehsPerHour="600" type="car"
    from="edgeWest" to="edgeEast"/>
</routes>
```

Estas definiciones permiten controlar la intensidad y dirección del tráfico, así como simular distintos tipos de vehículos (coches, autobuses, motocicletas, etc.). Tras ajustar la demanda, use `duarouter` para convertir los flujos en rutas específicas si fuera necesario.

4. **Actualizar los archivos de configuración.** Una vez depurada la red y configurados los semáforos y la demanda, asegúrese de que el archivo de configuración `.sumocfg` referencia correctamente todos los ficheros relevantes: `.net.xml`, `.rou.xml` y, en su caso, `.add.xml`. Ejecute una simulación breve para verificar que el escenario se carga sin mensajes de error.

Herramientas útiles

Para tareas de limpieza y configuración puede emplear las siguientes herramientas que acompañan a SUMO:

- **netedit:** editor gráfico que permite crear, modificar y depurar redes, así como editar semáforos, conexiones y restricciones de circulación. Resulta idóneo para eliminar segmentos irrelevantes y ajustar sentidos de vía.
- **sumo-gui:** interfaz gráfica de simulación que permite visualizar el comportamiento de los vehículos y los semáforos en tiempo real. Es útil para validar la red depurada y detectar cuellos de botella o comportamientos anómalos.
- **duarouter:** herramienta para convertir definiciones de flujos (`<flow>`) en rutas concretas, realizando el cálculo del recorrido óptimo entre origen y destino. Es necesaria cuando se trabaja con demanda no uniforme basada en datos observados.

3.5 Importación de un cruce desde OSM

Una vez instalado SUMO, el siguiente paso consiste en importar la intersección o corredor de interés. Existen dos alternativas.

3.5.1 Asistente OSM (*osmWebWizard*)

1. Ejecute el script `osmWebWizard.py` desde la carpeta de herramientas de SUMO. El script abre un navegador donde se puede navegar sobre el mapa de OSM, seleccionar la zona deseada y generar el escenario automáticamente. El uso básico es:

```
python tools/osmWebWizard.py
```

2. Seleccione “Lima, Peru” en el buscador y ajuste el *bounding box* sobre la intersección de interés. Active la opción “Traffic lights” si desea que se importen los semáforos y pulse “Generate Scenario”. El asistente creará la red *.net.xml*, una demanda aleatoria y un fichero *.sumocfg*; además iniciará *sumo-gui* con el escenario resultante. Los archivos se guardan en un subdirectorio con marca temporal.
3. Revise la red en *sumo-gui* y, si detecta calles desconectadas o giros imposibles, ajuste el recorte y repita la generación hasta obtener una red consistente.

Para ilustrar este proceso, la Figura 3.6 muestra la selección del área en OSM y la red generada en SUMO. A la izquierda se observa el recuadro delimitado sobre el mapa en el asistente *osmWebWizard*, y a la derecha la red importada tal como aparece en *sumo-gui*.

0.48

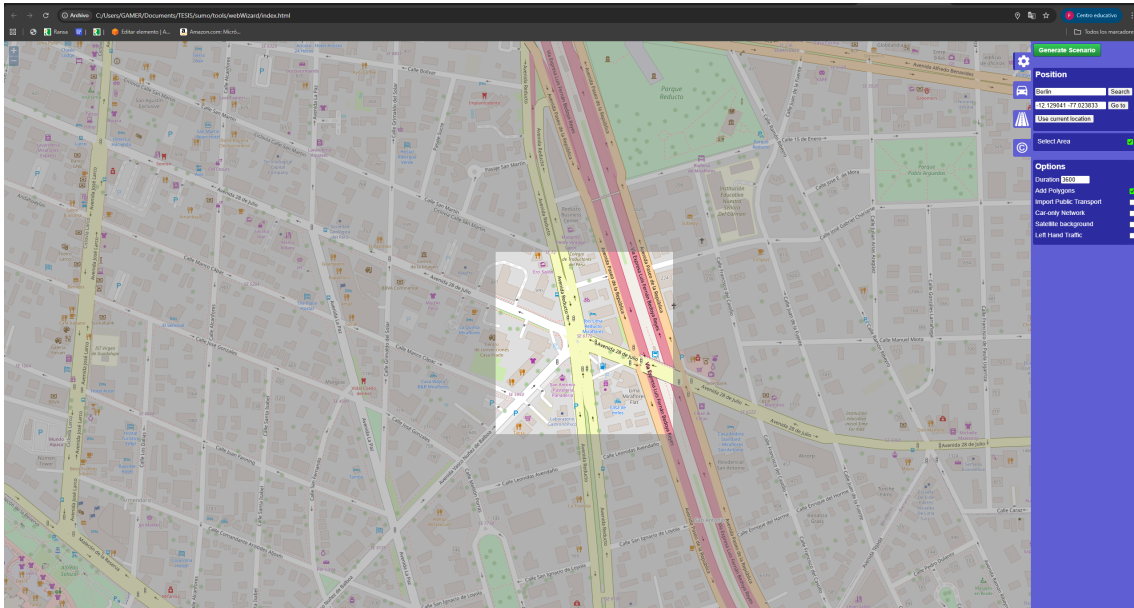


FIGURA 3.4: Selección del *bounding box* en OSM mediante el asistente *osmWeb-Wizard*.

0.48

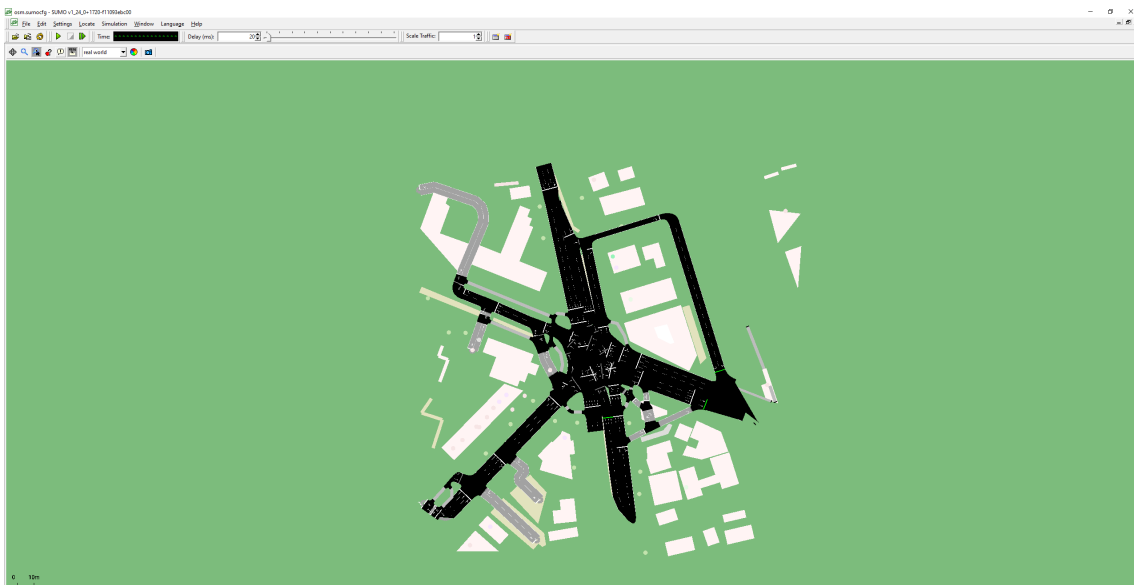


FIGURA 3.5: Red del cruce generada automáticamente en *sumo-gui*.

FIGURA 3.6: Importación de un cruce desde OSM a SUMO.

3.6 Generación y ejecución del escenario

3.6.1 Generar rutas de vehículos

Con la red validada, se requieren rutas de vehículos para simular el tráfico:

1. Para un escenario preliminar se puede recurrir a `randomTrips.py`. Este script selecciona aleatoriamente arcos de la red como origen y destino, distribuyendo los vehículos entre un tiempo de inicio `-b` y uno de fin `-e`. Se controla la intensidad del flujo con la opción `-p` y se garantiza reproducibilidad fijando una semilla con `--seed`. Un ejemplo básico que genera viajes durante 15 minutos con un vehículo por segundo es:

```
python tools/randomTrips.py -n lima.net.xml -b 0 -e 900 -p 1.0 \  
    --route-file rutas.rou.xml
```

2. Si se dispone de datos observados (aforos o tiempos de viaje), se pueden definir flujos origen-destino y enrutar con `duarouter`. Este proceso requiere calibrar tasas de arribo y cae fuera del alcance de este procedimiento básico.

3.6.2 Crear el archivo de configuración

Agrupe los archivos de red (`.net.xml`), rutas (`.rou.xml`) y, si procede, el programa de semáforos (`.add.xml`) en un único archivo de configuración `.sumocfg`. Este fichero referencia todos los elementos de entrada y define los intervalos de simulación.

3.6.3 Ejecución de SUMO

1. Para simular de forma interactiva, ejecute SUMO con interfaz gráfica:

```
sumo-gui -c escenario.sumocfg
```

2. Para simulaciones por lotes sin interfaz gráfica, utilice el ejecutable `sumo`:

```
sumo -c escenario.sumocfg
```

Los ejecutables de SUMO son aplicaciones de consola; las opciones `--net` y `--route-files` permiten especificar directamente los archivos de red y rutas si no se usa un `.sumocfg`.

3.7 Diagrama de flujo del módulo U1

La Figura 3.7 resume el procedimiento completo del módulo U1, desde la selección del cruce en Lima hasta la obtención del escenario SUMO listo para ser utilizado en el entrenamiento del agente de RL (U2).

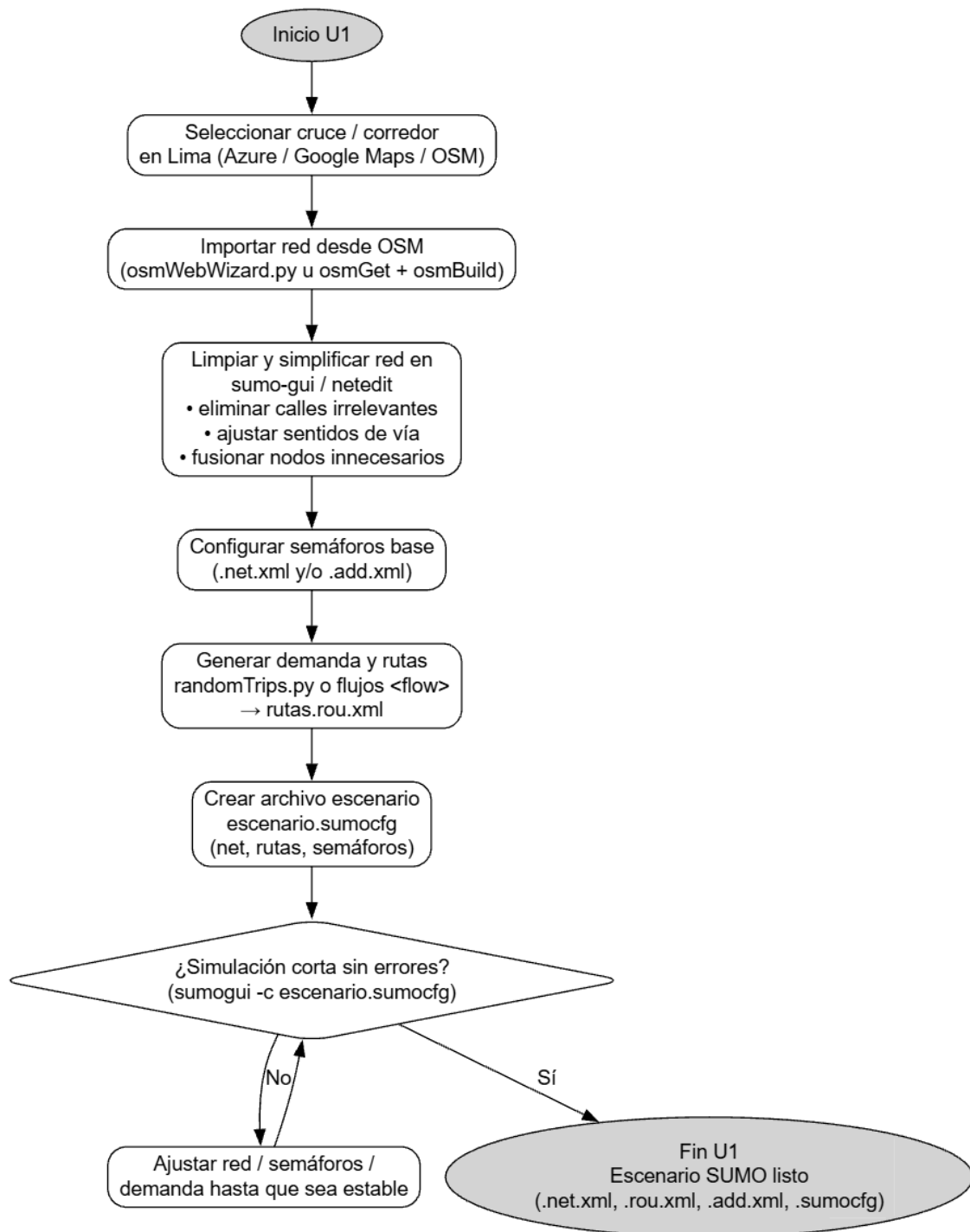


FIGURA 3.7: Diagrama de flujo del módulo U1: preprocesamiento y generación del escenario SUMO.

3.8 Resumen de las unidades

U1: Instalar y configurar SUMO; importar la intersección mediante el asistente OSM o descarga manual; limpiar y validar la red; generar la demanda vehicular y el archivo de configuración `.sumocfg`. El resultado es un escenario reproducible.

U2: Ejecutar el escenario con TraCI y entrenar un agente de aprendizaje por refuerzo que ajuste las fases semafóricas. U2 depende exclusivamente de los archivos generados en U1 y no modifica la red ni las rutas.

3.9 Desarrollo del módulo U2: Aprendizaje por Refuerzo para el control de semáforos

El segundo módulo de la metodología aborda el diseño e implementación de un controlador inteligente que modifique las fases de los semáforos en función del estado del tráfico. Para ello se recurre a técnicas de *Aprendizaje por Refuerzo* (*Reinforcement Learning*, RL), en las que un agente aprende a tomar decisiones mediante la interacción con un entorno simulado. Este apartado describe la formulación del problema, la interfaz con SUMO y los algoritmos de RL más habituales para el control de intersecciones.

Formulación del problema

En RL se define un *entorno* compuesto por un conjunto de *estados*, un conjunto de *acciones* y una función de *recompensa*. En el caso del control semafórico:

- **Estado:** resume la situación del tráfico en el cruce. Variables habituales incluyen la longitud de las colas por carril, el tiempo de espera acumulado, la

ocupación de los carriles o el tiempo restante de la fase actual. Estas mediciones pueden obtenerse en tiempo real a través de sensores virtuales como detectores de bucle o mediante la API de TraCI.

Para formalizar el estado en la implementación, se construye un vector s que concatena las métricas más relevantes. Por ejemplo, si la intersección tiene cuatro aproximaciones, una representación simple podría ser $s = (c_1, c_2, c_3, c_4, t)$, donde c_i es la longitud de la cola en la aproximación i y t es el tiempo transcurrido desde que se activó la fase actual. Esta abstracción reduce el espacio de estados a una dimensión manejable y captura las características que influyen en la congestión.

- **Acción:** representa la decisión del agente. En control semafórico se define normalmente como cambiar a la siguiente fase, mantener la fase actual durante unos segundos adicionales o seleccionar directamente una fase concreta de la tabla. Cada acción desencadena una actualización del semáforo en SUMO y, por tanto, modifica el flujo de vehículos.
- **Recompensa:** cuantifica la calidad de la acción realizada. Suele definirse como un valor negativo proporcional al tiempo total de espera, a la longitud de las colas o al número de vehículos detenidos. El objetivo del agente es maximizar la recompensa acumulada, lo que equivale a minimizar la congestión.

Una formulación típica de la recompensa r en el control semafórico es

$$r = -\left(w_1 \sum_i c_i + w_2 \sum_i w_i\right),$$

donde c_i es el número de vehículos en cola en la aproximación i , w_i es el tiempo de espera acumulado en esa cola y w_1, w_2 son pesos que permiten ponderar la importancia relativa de cada componente. Al ser negativa, la recompensa

penaliza estados con colas largas y tiempos de espera elevados, incentivando al agente a reducir ambos.

Con estos elementos se modela el control del semáforo como un *problema de decisión de Markov* (MDP). El agente observa el estado, aplica una acción, avanza la simulación y recibe una recompensa; a partir de la secuencia de experiencias va aprendiendo una política de control.

Interfaz con SUMO y ciclo de entrenamiento

Para implementar el bucle de RL se emplea la interfaz TraCI, proporcionada por SUMO, que permite controlar la simulación desde Python. Un ciclo típico de entrenamiento consta de los siguientes pasos:

1. **Inicialización:** cargar el escenario de U1 y establecer una semilla para garantizar reproducibilidad. Crear estructuras de datos para almacenar la política y la función de valor del agente.
2. **Obtención del estado:** en cada instante de control (p.ej., cada segundo) consultar a SUMO las variables de interés, como el número de vehículos por carril, los tiempos de espera o la fase actual del semáforo. TraCI proporciona funciones como `trafficlight.getPhase(tlID)` o `lane.getLastStepVehicleNumber(laneID)` para tal fin.
3. **Selección de la acción:** aplicar la política del agente (p.ej., ϵ -greedy para Q-learning o muestreo estocástico para algoritmos de política) para decidir si se mantiene la fase o se cambia a otra.

4. **Aplicación de la acción:** utilizar `trafficlight.setPhase(tlID, newPhase)` o `trafficlight.setPhaseDuration(tlID, duration)` para modificar el semáforo en SUMO. A continuación avanzar la simulación un número fijo de pasos (p.ej., 1 s) con `simulationStep`.
5. **Cálculo de la recompensa y actualización:** medir la recompensa inmediata (p.ej., el incremento del tiempo de espera total) y actualizar la estimación de la política o la función de valor según el algoritmo de RL elegido. Al finalizar cada episodio (un intervalo de simulación definido), almacenar estadísticas y, si procede, ajustar hiperparámetros.

Un fragmento representativo de código en Python que realiza este ciclo con TraCI y Q-learning es el siguiente:

```
import traci

# Iniciar SUMO con el escenario generado en U1
traci.start(["sumo", "-c", "escenario.sumocfg"])

# Identificador del semáforo que se controlará
tls_id = "tl_cruce"

for episode in range(num_episodes):
    # Restablecer la simulación al inicio del episodio
    traci.load(["-c", "escenario.sumocfg"])
    state = get_state(tls_id) # leer estado inicial
    done = False
    while not done:
        # Política epsilon-greedy: balance entre exploración y explotación
```

```

    action = select_action(Q, state, epsilon)
    apply_action(tls_id, action)                # cambiar fase en SUMO
    traci.simulationStep()                     # avanzar 1 s de simulación
    next_state = get_state(tls_id)
    reward = compute_reward(next_state)
    # Actualización Q-learning
    best_next = max(Q[next_state].values(), default=0.0)
    Q[state][action] += alpha * (reward + gamma * best_next - Q[state][action])
    state = next_state
    # criterio de finalización (p.ej., duración del episodio)
    done = check_termination()

# Finalizar la comunicación con SUMO
traci.close()

```

En este ejemplo simplificado, `get_state()` y `compute_reward()` encapsulan las llamadas a TraCI para obtener las métricas del entorno, y `apply_action()` utiliza `trafficlight.setPhase()` para aplicar la acción seleccionada. El entrenamiento puede adaptarse para implementar algoritmos más avanzados, por ejemplo sustituyendo la tabla Q por una red neuronal y utilizando técnicas de *Replay Buffer* y *target networks* (DQN), o aplicando métodos actor–critic (PPO, DDPG). En todos los casos, el bucle de interacción con SUMO sigue la misma estructura: observar el estado, decidir una acción, aplicarla a través de TraCI, avanzar la simulación y actualizar los parámetros del agente en función de la recompensa obtenida.

Este ciclo se repite durante múltiples episodios de entrenamiento hasta que la política converge o se alcanza un criterio de parada. Finalmente, al concluir cada episodio de entrenamiento se registran métricas de desempeño (tiempo medio de viaje, tamaño medio de cola, etc.) para evaluar la evolución del agente. Estos valores

se utilizan para ajustar los hiperparámetros y comparar distintas configuraciones de RL.

Algoritmos recomendados

Diversas técnicas de RL han sido aplicadas al control de semáforos. Los algoritmos tabulares, como *Q-learning* o *SARSA*, son adecuados para intersecciones sencillas con un número limitado de estados. Para escenarios de mayor complejidad se emplean métodos de *Deep Reinforcement Learning* como *Deep Q-Network* (DQN), *Proximal Policy Optimization* (PPO) o *Deep Deterministic Policy Gradients* (DDPG), que utilizan redes neuronales para aproximar la política y la función de valor. Estos algoritmos suelen implementarse con bibliotecas de aprendizaje automático (TensorFlow o PyTorch) y pueden integrarse con SUMO a través de entornos compatibles con Gym. La elección del algoritmo dependerá del tamaño del estado, la necesidad de cooperación entre intersecciones y los recursos computacionales disponibles.

Actualización de los valores Q y ejemplo de pseudocódigo

En el caso de algoritmos tabulares, como Q-learning, la política del agente se representa mediante una *tabla Q* en la que cada entrada $Q(s, a)$ almacena la expectativa de recompensa acumulada si se aplica la acción a en el estado s y se sigue la mejor política a partir de entonces. El proceso de aprendizaje consiste en actualizar iterativamente estos valores con información obtenida de las simulaciones. La regla de actualización de Q-learning se basa en el aprendizaje por diferencias temporales y se expresa de la siguiente manera:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)],$$

donde α es la tasa de aprendizaje, γ el factor de descuento, r la recompensa inmediata obtenida al ejecutar la acción a en el estado s , s' el estado siguiente y a' las posibles acciones en s' . Esta ecuación desplaza el valor Q actual en dirección del retorno observado ($r + \gamma \max_{a'} Q(s', a')$).

Para algoritmos *on-policy*, como SARSA, la actualización se realiza con la siguiente fórmula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)],$$

donde a' es la acción efectivamente seleccionada en el estado siguiente s' y α, γ tienen el mismo significado que en Q-learning. Al emplear el valor $Q(s', a')$ asociado a la política vigente en vez del máximo de todas las acciones, SARSA mantiene la coherencia entre exploración y actualización y suele conducir a comportamientos más conservadores.

Diagrama de flujo del módulo U2

La Figura 3.8 sintetiza el ciclo de entrenamiento del módulo U2. A partir del escenario generado en U1, el agente de RL interactúa con SUMO a través de TraCI, observa el estado del tráfico, selecciona acciones sobre el semáforo, actualiza sus valores $Q(s, a)$ (o los parámetros de la red neuronal) y registra métricas de desempeño por episodio.

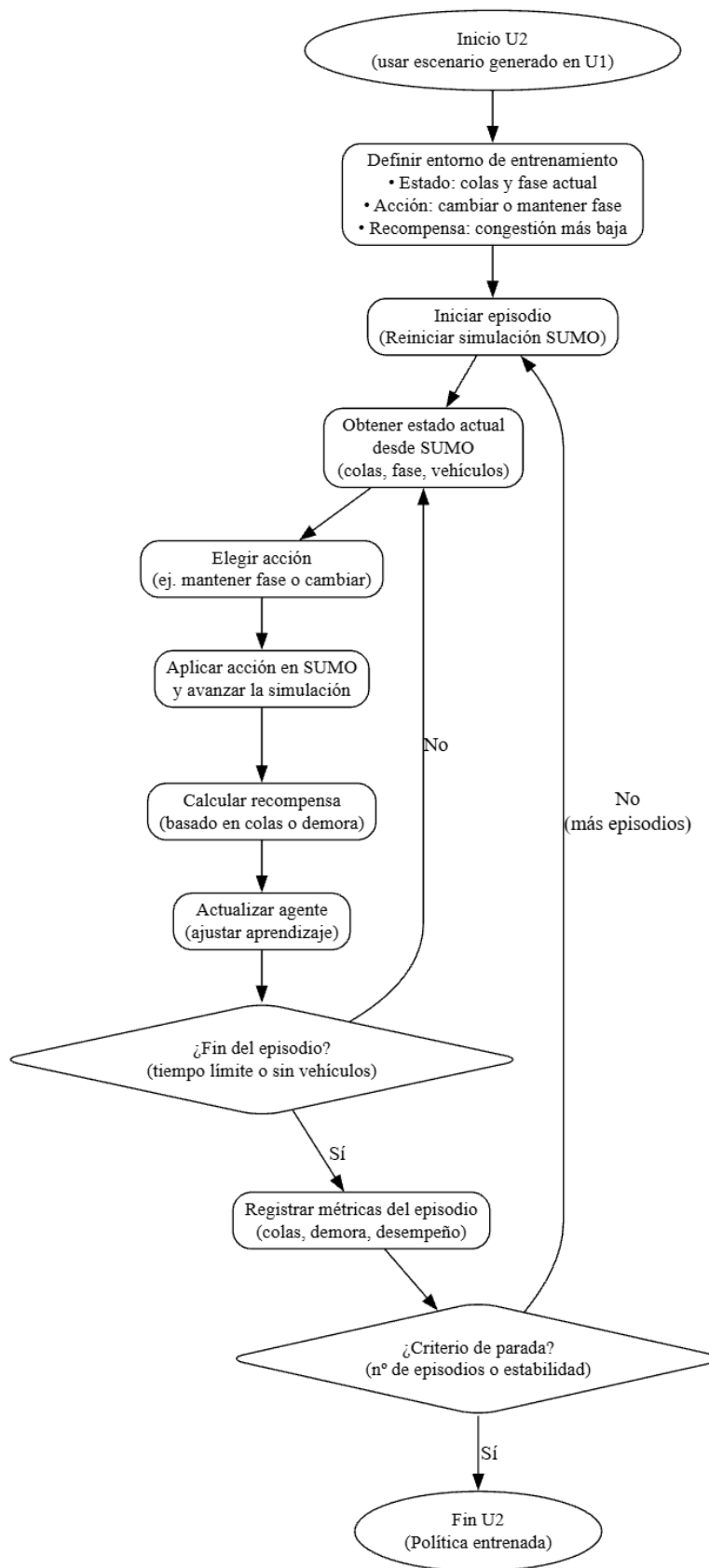


FIGURA 3.8: Diagrama de flujo del módulo U2: entrenamiento del agente de Aprendizaje por Refuerzo para el control semafórico.

CONCLUSIONES

RECOMENDACIONES

Lorem ipsum dolor sit amet, mei no laboramus gloriatur, no timeam euri-
pidis qui, eu eos aperiam patrioque accommodare. Mei probo accommodare an, ex
case minim salutatus sit. Ad doming impetus mei, placerat verterem has no. His
assum quaeque et, ea aliquid dissentiet eum. Copiosae evertitur cum te, sea clita
disputationi ea.

Libris ridens malorum cu vis. Per vitae eirmod cotidieque cu. Cum unum
mucius alterum eu, nam enim regione appellantur id. Cum putant dignissim te, est
dolore essent definiebas ei. Facilisi convenire vix in, dictas delenit et per. Pro tota
affert argumentum cu.

REFERENCIAS BIBLIOGRÁFICAS

- [1] J. D. C. Mass, “Semáforos inteligentes por medio de aprendizaje por refuerzo,” Trabajo de grado, Universidad de los Andes, Bogotá, Colombia, 2023, repositorio Uniandes.
- [2] S. Chakrabarti, “Reinforcement learning for traffic light optimization,” Master’s dissertation, Trinity College Dublin, School of Computer Science and Statistics, Dublin, Ireland, 2022.
- [3] J. C. C. Morales, “Predicción de tráfico vehicular aplicada a la optimización semafórica en un microsimulador,” Tesis de Maestría, Universidad Autónoma de Querétaro, Querétaro, México, 2024.
- [4] S. Project, “Importing networks from openstreetmap — sumo documentation,” <https://sumo.dlr.de/docs/Networks/Import/OpenStreetMap.html>, 2024, uso de `netconvert` para pasar OSM a `.net.xml`.
- [5] O. Foundation, “Openstreetmap wiki: About openstreetmap,” https://wiki.openstreetmap.org/wiki/About_OpenStreetMap, 2024, información general sobre OSM.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction (2nd ed.)*. MIT Press, 2018, capítulos 1–6 cubren definiciones, Q-learning y SARSA. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>

- [7] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. [Online]. Available: <https://www.nature.com/articles/nature14236>
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [9] U. D. of Transportation, “Traffic signal timing manual — chapter 3 (fhwa),” <https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter3.htm>, 2021, definición de *control delay* y su uso como Métrica de Efectividad.
- [10] —, “Definition, interpretation, and calculation of traffic analysis tools (fhwa),” <https://ops.fhwa.dot.gov/publications/fhwahop08054/execsum.htm>, 2021, cómo calcular delay, colas y paradas en evaluaciones.
- [11] *Highway Capacity Manual (HCM)*. Transportation Research Board, 2010, metodología de LOS y control delay en intersecciones. [Online]. Available: <https://www.jpautoceste.ba/wp-content/uploads/2022/05/Highway-Capacity-Manual-2010-PDFDrive-.pdf>
- [12] C. Yu *et al.*, “The surprising effectiveness of ppo in cooperative multi-agent games,” in *NeurIPS 2021 Deep RL Workshop*, 2021, presenta MAPPO y compara con IPPO como baseline. [Online]. Available: <https://arxiv.org/abs/2103.01955>
- [13] S. Project, “Traci — traffic control interface (python) — sumo documentation,” <https://sumo.dlr.de/docs/TraCI.html>, 2024, interfaz cliente-servidor para controlar SUMO en tiempo de simulación.
- [14] P. Á. López, M. Behrisch *et al.*, “Microscopic traffic simulation using sumo,” <https://elib.dlr.de/127994/1/simulative.pdf>, 2018, descripción general de SUMO y su ecosistema, incl. TraCI.

- [15] S. Project, “Emissions — sumo documentation,” <https://sumo.dlr.de/docs/Tools/Emissions.html>, 2024, modelos HBEFA y PHEMlight; herramientas de cálculo.
- [16] S.-B. Developers, “Stable-baselines3 documentation,” <https://stable-baselines3.readthedocs.io/>, 2024, implementaciones de PPO, DQN, A2C en PyTorch.

ANEXOS

Los algoritmos desarrollados